

International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

HTTP Analysis Using Wireshark: Embedded Image Traffic

Student Name: Kafayat Omolara Animashawun

Student ID: 2025/FWSD/11317

Instructor Name: Aminu Idris

Date of Submission: 10/09/2026

EXECUTIVE SUMMARY

This practical exercise demonstrates the capture and forensic analysis of plaintext HTTP traffic generated when a web browser retrieves an HTML webpage containing an embedded JPEG image.

A controlled local laboratory environment was established using Kali Linux, Apache2, Wireshark, TShark, curl, ImageMagick, and standard Linux file-verification utilities. The webpage and embedded image were hosted on the local Apache server and accessed through the browser using the loopback address 127.0.0.1.

The analysis established that the browser did not retrieve the webpage and image as a single object. Instead, the browser first requested /image.html and subsequently generated a separate HTTP GET request for /lab_photo.jpg. Wireshark identified successful HTTP responses for both resources, including a 200 OK response for the JPEG image.

The JPEG response was also shown by Wireshark to involve TCP reassembly. The HTTP response was reconstructed from two TCP segments contributing 32,768 bytes and 2,809 bytes respectively, producing a reassembled TCP payload of 35,577 bytes.

The original JPEG was independently verified before transmission and assigned a SHA-256 baseline hash. The recovered object exported from the packet capture was subsequently intended to be compared against this baseline to establish its integrity.

The exercise also examines the difference between browser and curl behaviour. A browser processes the returned HTML and automatically requests referenced resources

such as an embedded image, whereas a basic curl request retrieves only the explicitly requested URL.

PURPOSE OF THE ASSIGNMENT

The purpose of this practical is to demonstrate the forensic analysis of plaintext HTTP traffic using Wireshark. The exercise focuses on understanding how a browser retrieves an HTML webpage and subsequently requests an embedded image as a separate HTTP object.

The practical also demonstrates how HTTP data is transported over TCP, how Wireshark reconstructs segmented traffic, and how an object can be recovered from a packet capture and verified against its original source using a SHA-256 cryptographic hash.

OBJECTIVES OF THE ASSIGNMENT

The objectives of this practical assignment are to:

1. Capture and analyze plaintext HTTP traffic generated when a browser retrieves an HTML webpage containing an embedded image.
2. Identify separate HTTP GET requests for the HTML document and embedded JPEG image, analyze their corresponding server responses, demonstrate TCP segmentation and reassembly, recover the transferred image from the packet capture, and verify its integrity using SHA-256.
3. Compare browser-generated HTTP behaviour with a direct curl request to demonstrate how browsers automatically retrieve embedded resources while curl normally retrieves only the explicitly requested resource.

METHODOLOGY OF THE ASSIGNMENT

The practical was performed within an isolated Kali Linux laboratory environment using the local loopback interface. A dedicated directory structure was created to separate evidence, working files, exported objects, screenshots, scripts, and reports.

A JPEG image was selected as the original source object and independently verified before being introduced into the HTTP environment. Its file type, dimensions, size, and SHA-256 hash were recorded as baseline information.

An HTML webpage was then created containing an element referencing the JPEG. Both objects were deployed to the Apache document root and tested using curl to confirm that they were available through plaintext HTTP.

The webpage was subsequently accessed through a browser while TShark captured traffic on the loopback interface using a TCP port 80 capture filter. The resulting PCAPNG file was opened in Wireshark and examined at the HTTP and TCP layers.

The analysis focused on identifying:

- The HTML GET request.
- The HTML HTTP response.
- The separate JPEG GET request.
- The JPEG HTTP response.
- TCP segmentation and reassembly.
- HTTP object export and recovery.
- SHA-256 integrity verification.
- Differences between browser and curl behaviour.

All network traffic analysed during the practical was generated within the local laboratory environment using 127.0.0.1.

TOOLS AND RESOURCES USED

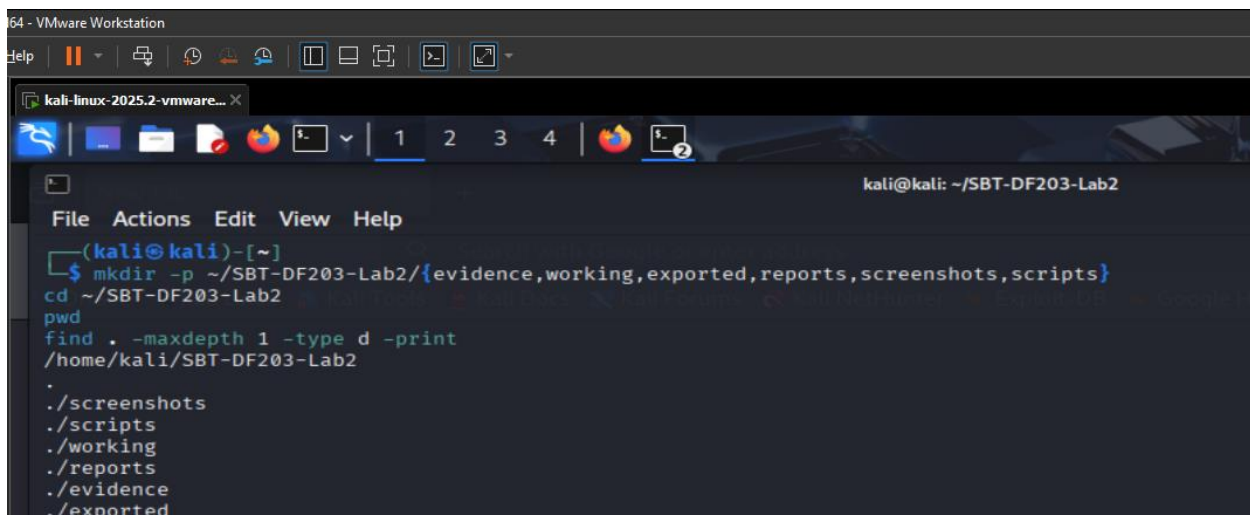
The following tools and resources were used during the practical exercise:

Tool / Resource	Purpose
Kali Linux	Laboratory operating system
Apache2 2.4.68	Local HTTP web server
Wireshark 4.6.6	Graphical packet capture analysis
TShark 4.6.6	Command-line packet capture
curl 8.21.0	HTTP availability and browser-behaviour comparison
ImageMagick 7.1.2-27	Image verification

file utility 5.47	File-type identification
SHA-256	Cryptographic integrity verification
lab_photo.jpg	Original embedded image
image.html	Laboratory webpage
browser_http_capture.pcapng	Captured browser HTTP evidence

Laboratory Environment Preparation

The practical commenced by establishing a dedicated working environment for Lab 2. A separate directory was created to ensure that all artefacts generated during the investigation could be organised and preserved appropriately. The directory contains separate locations for captured network evidence, working files, exported HTTP objects, screenshots, scripts, and the final report. The resulting directory was then verified to confirm that the required folders had been created successfully.



```

kali@kali: ~/SBT-DF203-Lab2
$ mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}
$ cd ~/SBT-DF203-Lab2
$ pwd
/home/kali/SBT-DF203-Lab2
$ find . -maxdepth 1 -type d -print
.
./screenshots
./scripts
./working
./reports
./evidence
./exported

```

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ sudo apt update
sudo apt install -y apache2 curl wireshark tshark imagemagick file
[sudo] password for kali:
Get:1 https://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 https://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:3 https://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.7 MB]
Get:4 https://kali.download/kali kali-rolling/contrib amd64 Packages [113 kB]
Get:5 https://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:6 https://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Fetched 76.4 MB in 31s (2,479 kB/s)
2477 packages can be upgraded. Run 'apt list --upgradable' to see them.
apache2 is already the newest version (2.4.68-1).
curl is already the newest version (8.21.0-2).
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
tshark set to manually installed.
The following packages were automatically installed and are no longer required:
aspnetcore-runtime-6.0 libice-dev libx11-dev
aspnetcore-targeting-pack-6.0 libkadm5clnt-mit12 libxar1
bloodhound.py libkadm5srv-mit12 libxau-dev
dotnet-apphost-pack-6.0 libkdb5-10t64 libxcb1-dev
dotnet-host liblvm18 libxdmcp-dev
dotnet-hostfxr-6.0 liblvm2pirvlib18.1 libxext-dev
dotnet-runtime-6.0 liblz4-dev libxmp4
dotnet-runtime-deps-6.0 liblzma-dev libxshash-dev
dotnet-sdk-6.0 libmagic-dev libz3-dev
dotnet-targeting-pack-6.0 libmbcCrypto16 libzix3
hyphen-en-us libmjpegutils-2.1-0t64 linux-libc-dev
python3-asynsocks python3-pyppkatz
python3-bitstruct python3-pyqt6.sip
python3-cssselect2 python3-pyvinc
python3-docopt python3-secretsocks
python3-donut python3-sqlalchemy-utc
python3-dploit python3-stix2
python3-dropbox python3-stix2-patterns
python3-humanize python3-stone
python3-jq python3-terminaltables
python3-jwcrypto python3-unicrypto
python3-libnmap python3-websocket
```

Verification of Required Laboratory Tools

Before generating network traffic, the tools required for the practical were verified. Apache2 is required to host the local webpage and embedded image, while Wireshark and TShark will be used to capture and analyse the HTTP traffic. curl will later be used to demonstrate the difference between command-line retrieval and browser-generated requests. ImageMagick and the file utility will support verification of the recovered image.

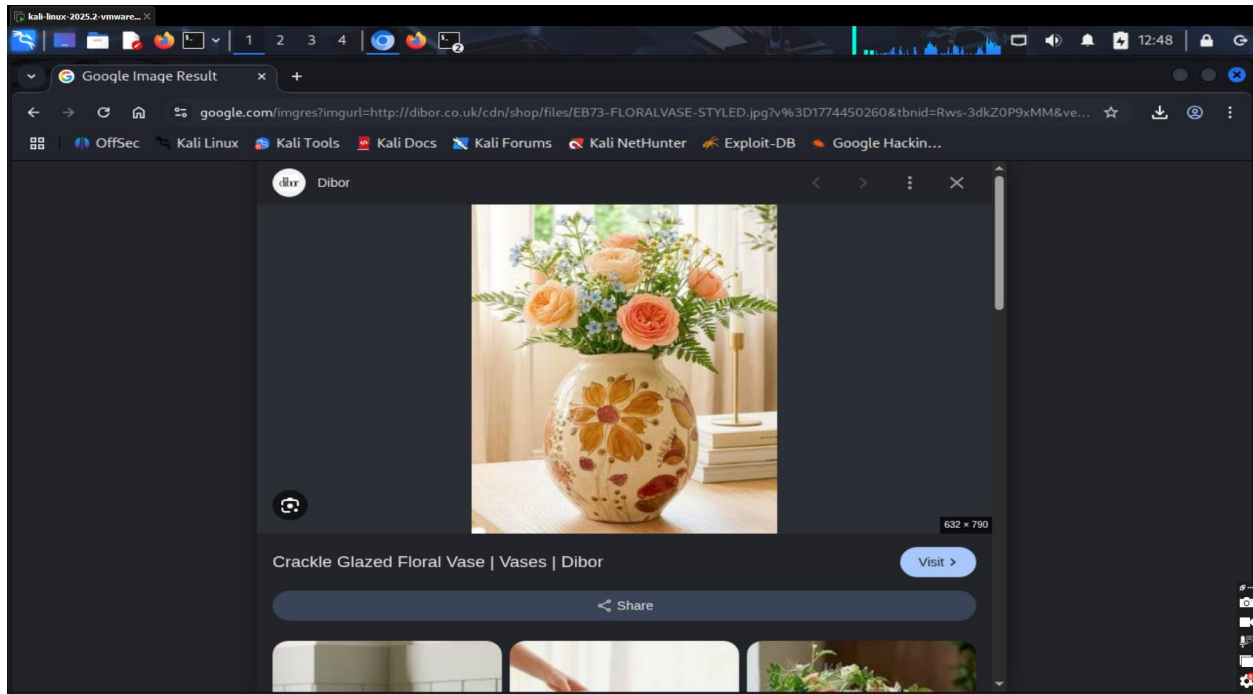
The installed versions were checked using the following:

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ sudo restart
sudo: restart: command not found

(kali@kali)-[~/SBT-DF203-Lab2]
$ apache2 -v
tshark --version | head -n 2
wireshark --version | head -n 2
curl --version | head -n 1
identify -version | head -n 1
file --version | head -n 1
Server version: Apache/2.4.68 (Debian)
Server built: 2026-06-11T18:22:17
Warning: program compiled against libxml 215 using older 214
TShark (Wireshark) 4.6.6.
```

Source Image Acquisition

The practical requires an image object to be embedded within the locally hosted HTML page and transferred over plaintext HTTP. A flower-vase image was selected as the test object for the laboratory exercise. A dedicated working/source directory was created to preserve the original source object separately from subsequently recovered network evidence.



The image was downloaded directly into the laboratory workspace and will be treated as the **original source object**. Before generating the HTTP traffic, the file will be examined to confirm its file type and dimensions and subsequently hashed using SHA-256. This baseline hash will later be compared with the hash of the image recovered from the packet capture.

```
kali@kali: ~/SBT-DF203-Lab2
File Actions Edit View Help
Version: ImageMagick 7.1.2-27 Q16 x86_64 20260705 https://imagemagick.org
file-5.47

(kali@kali)~[~/SBT-DF203-Lab2]
$ cd ~/SBT-DF203-Lab2
mkdir -p working/source

(kali@kali)~[~/SBT-DF203-Lab2]
$ wget -O working/source/lab_photo.jpg "http://dibor.co.uk/cdn/shop/files/EB73-FLORALVASE-STYLED.jpg?v=1774450260"
--2026-09-10 12:43:47-- http://dibor.co.uk/cdn/shop/files/EB73-FLORALVASE-STYLED.jpg?v=1774450260
Resolving dibor.co.uk (dibor.co.uk) ... 23.227.38.65
Connecting to dibor.co.uk (dibor.co.uk)[23.227.38.65]:80 ... failed: Connection refused.

(kali@kali)~[~/SBT-DF203-Lab2]
$ ls
evidence exported reports screenshots scripts working

(kali@kali)~[~/SBT-DF203-Lab2]
$ cd working

(kali@kali)~[~/SBT-DF203-Lab2/working]
$ ls
source

(kali@kali)~[~/SBT-DF203-Lab2/working]
$ cd source

(kali@kali)~[~/SBT-DF203-Lab2/working/source]
$ s
s: command not found

(kali@kali)~[~/SBT-DF203-Lab2/working/source]
$ ls
lab_photo.jpg

(kali@kali)~[~/SBT-DF203-Lab2/working/source]
$ cd ..

(kali@kali)~[~/SBT-DF203-Lab2/working]
$ cd ..

(kali@kali)~[~/SBT-DF203-Lab2]
$
```

Source Image Verification and Hashing

The selected image was placed in the dedicated working/source directory and established as the original source object for the practical. Before the image was introduced into the HTTP environment, its file type, dimensions, colour characteristics and file size were verified using standard Linux forensic and file-identification utilities.

The verification confirmed that lab_photo.jpg is a valid JPEG image with dimensions of 399 × 501 pixels and an approximate file size of 35 KB. This establishes the characteristics of the original object that will subsequently be served by the local Apache web server.

```
(kali@kali)~[~/SBT-DF203-Lab2/working/source]
$ ls
lab_photo.jpg
(kali@kali)~[~/SBT-DF203-Lab2/working/source]
$ cd ..
(kali@kali)~[~/SBT-DF203-Lab2/working]
$ cd ..
(kali@kali)~[~/SBT-DF203-Lab2]
$ cd ~/SBT-DF203-Lab2
file working/source/lab_photo.jpg
identify working/source/lab_photo.jpg
ls -ln working/source/lab_photo.jpg
sha256sum working/source/lab_photo.jpg
working/source/lab_photo.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 399x501, component
s 3
working/source/lab_photo.jpg JPEG 399x501 399x501+0+0 8-bit sRGB 352898 0.000u 0:00.000
-rw-rw-r-- 1 kali kali 35K Sep 10 12:45 working/source/lab_photo.jpg
25f4073c345856b582500b9a6f59b38248984081b2dac90e25a54270113ad8ce working/source/lab_photo.jpg
(kali@kali)~[~/SBT-DF203-Lab2]
$
```

A SHA-256 cryptographic hash was then calculated for the original image. The resulting digest,

25f4073c345856b582500b9a6f59b38248984081b2dac90e25a54270113ad8ce

was recorded as the baseline integrity value. This value will later be compared with the SHA-256 hash of the image recovered from the HTTP packet capture. A matching hash will provide evidence that the recovered image is identical to the original source object.

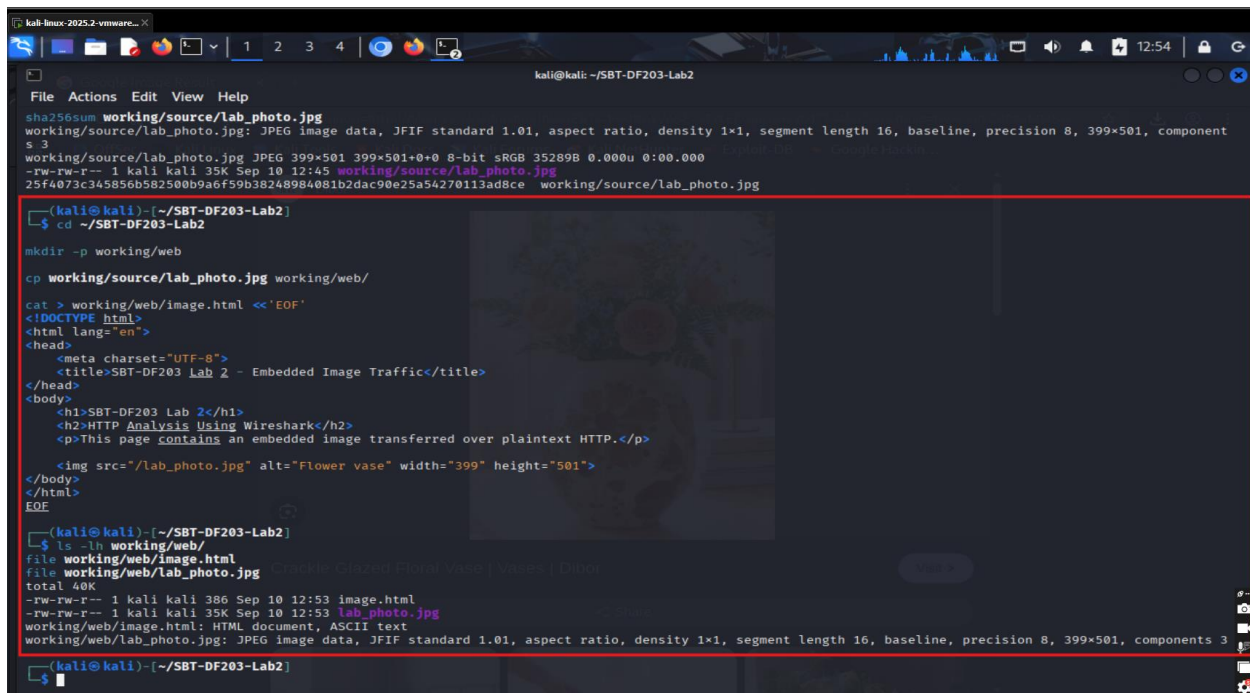
Creation of the HTTP Test Objects

After establishing the original image and its SHA-256 baseline, the web content required for the HTTP traffic experiment was prepared. A dedicated working/web directory was created to contain the webpage and the image that will be served by the local Apache web server.

The HTML document was created with a reference to lab_photo.jpg using an HTML element. This configuration is intentional because the browser must first request the

HTML document and then request the embedded image as a separate web object. Consequently, the resulting packet capture should contain distinct HTTP transactions for the webpage and the image.

The source image was copied into the same web directory and independently verified to ensure that the object referenced by the HTML document is a valid JPEG file. The HTML document and image were therefore confirmed as the two source objects that will participate in the HTTP exchange.



```
kali@kali: ~/SBT-DF203-Lab2
File Actions Edit View Help
sha256sum working/source/lab_photo.jpg
working/source/lab_photo.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 399x501, component
s 3
working/source/lab_photo.jpg JPEG 399x501 399x501+0+0 8-bit sRGB 352898 0.000u 0:00.000
-rw-rw-r-- 1 kali kali 35K Sep 10 12:45 working/source/lab_photo.jpg
25f4073c345856b582500b9a6f59b38248984081b2dac90e25a54270113ad8ce working/source/lab_photo.jpg

(kali@kali)~[~/SBT-DF203-Lab2]
$ cd ~/SBT-DF203-Lab2

mkdir -p working/web
cp working/source/lab_photo.jpg working/web/

cat > working/web/image.html <<'EOF'
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>SBT-DF203 Lab 2 - Embedded Image Traffic</title>
</head>
<body>
<h1>SBT-DF203 Lab 2</h1>
<h2>HTTP Analysis Using Wireshark</h2>
<p>This page contains an embedded image transferred over plaintext HTTP.</p>

</body>
</html>
EOF

(kali@kali)~[~/SBT-DF203-Lab2]
$ ls -lh working/web/
file working/web/image.html 1.2K HTML document, ASCII text
file working/web/lab_photo.jpg 35K JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 399x501, components 3
total 40K
-rw-rw-r-- 1 kali kali 386 Sep 10 12:53 image.html
-rw-rw-r-- 1 kali kali 35K Sep 10 12:53 lab_photo.jpg
working/web/image.html: HTML document, ASCII text
working/web/lab_photo.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 399x501, components 3
```

Verification of the Apache Web Server

Before deploying the laboratory web objects, the Apache HTTP server was checked to confirm that the local web service was active and ready to host the test webpage. The service status showed that Apache was active (running), confirming that the local HTTP service was operational within the Kali Linux laboratory environment.

The Apache document root was then inspected to establish its initial state before introducing the Lab 2 files. The inspection showed the existing contents of /var/www/html/, including the pre-existing basic.html, index.html, index.nginx-debian.html, and dvwa directory.

This preliminary verification was performed to ensure that the Lab 2 files could be introduced without unnecessarily modifying or deleting existing web-server content.

```
(kali@kali) [~/SBT-DF203-Lab2]
$ sudo systemctl status apache2 --no-pager
[sudo] password for kali:
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-10 12:29:45 EDT; 25min ago
     Invocation: 6b96a30211c54888bdec2c678a281a09
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 630104 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 6 (limit: 8113)
       Memory: 38.2M (peak: 38.5M)
          CPU: 768ms
       CGroup: /system.slice/apache2.service
               └─630104 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─630117 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─630118 /usr/sbin/apache2 -k start -DFOREGROUND
                     └─630119 /usr/sbin/apache2 -k start -DFOREGROUND
                       └─630121 /usr/sbin/apache2 -k start -DFOREGROUND
                         └─630122 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 10 12:29:44 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 10 12:29:44 kali apachectl[630104]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1; this message
Sep 10 12:29:45 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.

(kali@kali) [~/SBT-DF203-Lab2]
$ ls -ls /var/www/html/
total 32
drwxr-xr-x 3 root root 4096 Sep  8 14:24 .
drwxr-xr-x 3 root root 4096 May 29 2025 ..
-rw-r--r-- 1 root root 257 Sep  8 14:24 basic.html
drwxr-xr-x 12 www-data www-data 4096 Nov 19 2025 dvwa
-rw-r--r-- 1 root root 10703 May 29 2025 index.html
-rw-r--r-- 1 root root 615 May 29 2025 index.nginx-debian.html
```

Deployment of the Lab 2 Web Objects

Following verification of the Apache service and its initial document root, the two prepared laboratory objects were deployed to /var/www/html/. The HTML page and embedded JPEG image were copied into the Apache document root so they could be served through the local HTTP service.

The deployed files were then checked using ls -lh. The output confirms that image.html is 386 bytes and lab_photo.jpg is approximately 35 KB, matching the prepared laboratory files. This establishes the two HTTP resources that will subsequently be requested by the browser: the HTML document first, followed by the embedded image.

```
(kali@kali) [~/SBT-DF203-Lab2]
$ cd ~/SBT-DF203-Lab2

$ sudo cp working/web/image.html /var/www/html/
$ sudo cp working/web/lab_photo.jpg /var/www/html/

(kali@kali) [~/SBT-DF203-Lab2]
$ ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 386 Sep 10 12:57 /var/www/html/image.html
-rw-r--r-- 1 root root 35K Sep 10 12:57 /var/www/html/lab_photo.jpg

(kali@kali) [~/SBT-DF203-Lab2]
$
```

The prepared image.html webpage and lab_photo.jpg image were subsequently copied into Apache's document root. This makes both objects available through the local HTTP

service and establishes the environment required to generate the browser-based HTTP traffic for the investigation.

The use of a locally hosted Apache server ensures that the traffic generated during the exercise remains within the authorised laboratory environment rather than targeting an external or production system.

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ cd ~/SBT-DF203-Lab2

sudo cp working/web/image.html /var/www/html/
sudo cp working/web/lab_photo.jpg /var/www/html/

(kali@kali)-[~/SBT-DF203-Lab2]
$ ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 386 Sep 10 12:57 /var/www/html/image.html
-rw-r--r-- 1 root root 35K Sep 10 12:57 /var/www/html/lab_photo.jpg

(kali@kali)-[~/SBT-DF203-Lab2]
$
```

Verification of HTTP Object Availability

Before beginning packet capture, the deployed resources were tested directly through the local Apache server using curl. This step confirms that the web server can successfully retrieve both the HTML document and the embedded JPEG image.

The response for image.html returned HTTP/1.1 200 OK, with a content length of 386 bytes and content type text/html. The image request also returned HTTP/1.1 200 OK, with a content length of 35,289 bytes and content type image/jpeg.

The results establish that both resources are correctly deployed and available through plaintext HTTP. The matching content lengths also provide useful reference values for the subsequent Wireshark analysis and recovered-object verification.

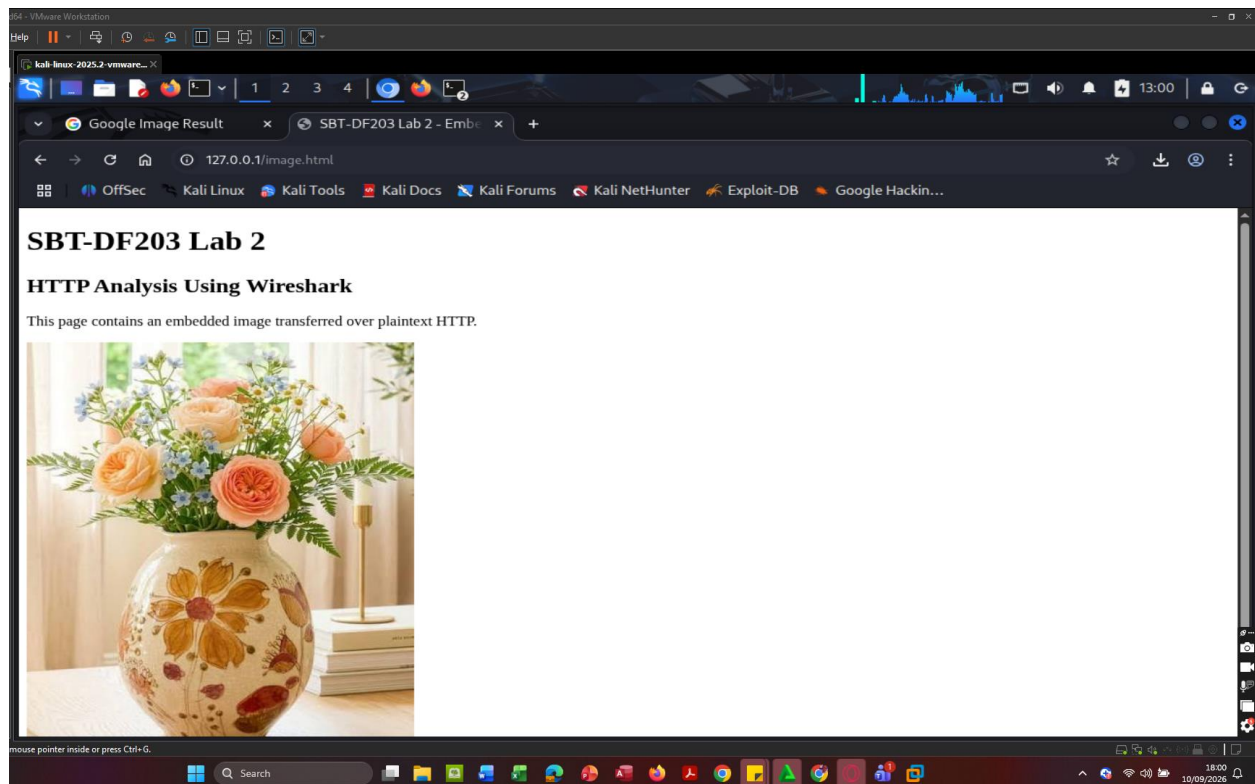
```
(kali@kali)-[~/SBT-DF203-Lab2]
$ curl -I http://127.0.0.1/image.html
curl -I http://127.0.0.1/lab_photo.jpg
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 16:59:40 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 16:57:11 GMT
ETag: "182-65b23dbeb5923"
Accept-Ranges: bytes
Content-Length: 386
Vary: Accept-Encoding
Content-Type: text/html

HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 16:59:40 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 16:57:11 GMT
ETag: "89d9-65b23dbebb689"
Accept-Ranges: bytes
Content-Length: 35289
Content-Type: image/jpeg
```

Browser Verification of the HTTP Test Page

After confirming that both the HTML document and JPEG image were available through Apache using curl, the test page was accessed through a web browser using the local HTTP address: <http://127.0.0.1/image.html>. The page loaded successfully and displayed the SBT-DF203 Lab 2 heading, accompanying text, and the embedded flower-vase image. This confirms that the HTML document was successfully retrieved by the browser and that the element referenced a valid image resource that could also be retrieved from the Apache server.

This browser-based verification is important because the practical requires analysis of browser-generated HTTP traffic, rather than relying only on command-line requests. When the page is loaded, the browser is expected to generate separate HTTP requests for the HTML document and the embedded JPEG object. These requests will be captured and examined in the subsequent Wireshark analysis.



Packet Capture in Progress

After the browser was refreshed while TShark was actively capturing traffic, the webpage and embedded image were successfully loaded. The capture was then manually stopped using Ctrl+C after the browser transaction had completed.

TShark reported 78 captured packets before the capture was stopped. The ^C displayed in the terminal indicates that the capture was intentionally terminated by the analyst using the keyboard interrupt command. This does not indicate an error or failed capture.

The captured traffic was written to the temporary evidence file:

```
/tmp/browser_http_capture.pcapng
```

The resulting PCAP will now be examined in Wireshark to determine whether the browser-generated HTTP conversation contains the expected HTML and embedded JPEG requests, together with their corresponding server responses.

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ sudo tshark -i lo -f "tcp port 80" -w evidence/browser_http_capture.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
tshark: The file to which the capture would be saved ("evidence/browser_http_capture.pcapng") could not be opened: Permission denied.

(kali@kali)-[~/SBT-DF203-Lab2]
$ cd ~/SBT-DF203-Lab2

sudo tshark -i lo -f "tcp port 80" -w /tmp/browser_http_capture.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
78 ^C

(kali@kali)-[~/SBT-DF203-Lab2]
$
```

Verification of the Captured Evidence File

Following completion of the browser traffic capture, the PCAP was moved from the temporary capture location into the dedicated evidence directory. Ownership was also assigned to the kali user to ensure that the captured evidence can be accessed during subsequent analysis without requiring unnecessary privilege escalation.

The file verification confirms that browser_http_capture.pcapng was successfully created with a size of 100 KB. This establishes the captured network traffic as the evidence file that will be analysed in Wireshark.

The capture contains the traffic generated during the browser's fresh retrieval of the laboratory webpage and its embedded image. The next stage is to examine this capture at the packet level and identify the individual HTTP requests and responses.

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ cd ~/SBT-DF203-Lab2

sudo tshark -i lo -f "tcp port 80" -w /tmp/browser_http_capture.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
78 ^C

(kali@kali)-[~/SBT-DF203-Lab2]
$ cd ~/SBT-DF203-Lab2

sudo mv /tmp/browser_http_capture.pcapng evidence/browser_http_capture.pcapng
sudo chown kali:kali evidence/browser_http_capture.pcapng

ls -lh evidence/browser_http_capture.pcapng
-rw-r--r-- 1 kali kali 100K Sep 10 13:08 evidence/browser_http_capture.pcapng

(kali@kali)-[~/SBT-DF203-Lab2]
$
```

Initial Analysis of the Browser Capture

The completed PCAP was opened in Wireshark for packet-level analysis. The capture contains 78 packets, all associated with communication between 127.0.0.1, confirming that the recorded traffic occurred between the browser and the locally hosted Apache web server.

The packet list shows the establishment of a TCP connection from the browser's ephemeral port 60384 to the Apache HTTP service on destination port **80**. Following the

TCP connection establishment, Wireshark identifies the browser's HTTP requests and the corresponding server responses.

Most importantly, the capture demonstrates the retrieval of multiple web objects. The browser first issued:

GET /image.html HTTP/1.1

The Apache server responded with:

HTTP/1.1 200 OK (text/html)

The browser then issued a separate request:

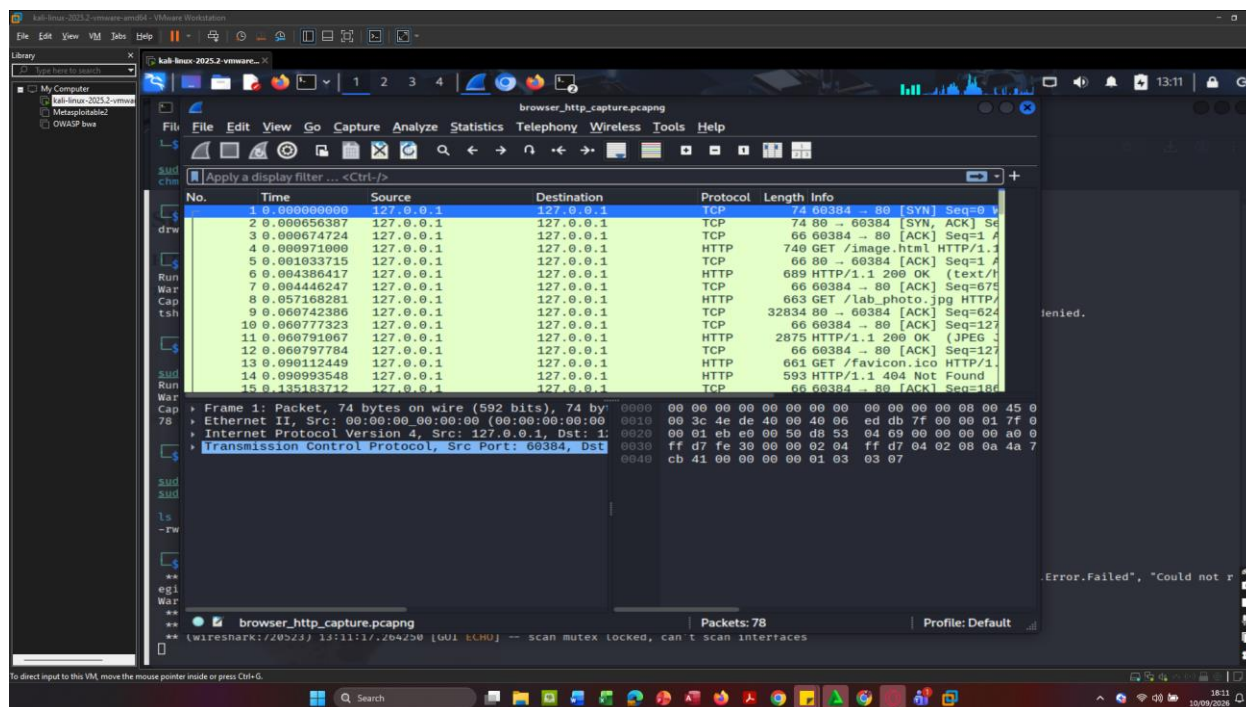
GET /lab_photo.jpg HTTP/...

and Wireshark identifies the corresponding response as:

HTTP/1.1 200 OK (JPEG...)

The capture also contains a subsequent request for /favicon.ico, which received a 404 Not Found response. This is normal browser behaviour and is not one of the required laboratory objects. We will distinguish it from the relevant HTML and JPEG traffic during analysis.

This is significant evidence for the practical because it demonstrates that the webpage and embedded image were not transferred as a single object. The browser retrieved the HTML document first and subsequently generated another HTTP request for the JPEG referenced by the HTML element.



Identification of Browser HTTP Requests

To isolate application-layer requests from the captured traffic, the Wireshark display filter `http.request` was applied. This filter restricts the packet list to HTTP request messages generated by the browser.

The filtered results contain separate requests for the laboratory webpage and its embedded image. Wireshark identifies requests for `GET /image.html HTTP/1.1` as well as `GET /lab_photo.jpg`, demonstrating that the browser retrieved the webpage and image as distinct HTTP resources.

The filtered capture also contains requests for `/favicon.ico`. These requests are incidental browser behaviour and are not part of the laboratory web objects under investigation. They are therefore excluded from the primary analysis.

The presence of distinct GET requests for `/image.html` and `/lab_photo.jpg` provides direct packet-level evidence that the embedded image was requested separately after the HTML document was retrieved.

Wireshark interface showing a list of HTTP requests. The filter is `http.request`. The list shows various GET requests for images and favicons. The packet details pane shows the structure of the selected packet.

No.	Time	Source	Destination	Protocol	Length	Info
4	0.000971000	127.0.0.1	127.0.0.1	HTTP	740	GET /image.html HTTP/1.1
8	0.057168281	127.0.0.1	127.0.0.1	HTTP	663	GET /lab_photo.jpg HTTP/1.1
13	0.090112449	127.0.0.1	127.0.0.1	HTTP	661	GET /favicon.ico HTTP/1.1
22	23.670118801	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
26	23.749035992	127.0.0.1	127.0.0.1	HTTP	618	GET /favicon.ico HTTP/1.1
29	27.821825348	127.0.0.1	127.0.0.1	HTTP	740	GET /image.html HTTP/1.1
32	27.849732138	127.0.0.1	127.0.0.1	HTTP	663	GET /lab_photo.jpg HTTP/1.1
36	27.886963517	127.0.0.1	127.0.0.1	HTTP	661	GET /favicon.ico HTTP/1.1
46	71.904047774	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
50	72.714569363	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
53	72.906591616	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
56	73.090755929	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
59	73.256823439	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
62	73.459922060	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
65	73.596896231	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1

Request: Boolean Packets: 78 · Displayed: 18 (23.1%) Profile: Default

Wireshark interface showing a list of HTTP requests. The filter is `http.request`. The list shows various GET requests for images and favicons. The packet details pane shows the structure of the selected packet.

No.	Time	Source	Destination	Protocol	Length	Info
4	0.000971000	127.0.0.1	127.0.0.1	HTTP	740	GET /image.html HTTP/1.1
8	0.057168281	127.0.0.1	127.0.0.1	HTTP	663	GET /lab_photo.jpg HTTP/1.1
13	0.090112449	127.0.0.1	127.0.0.1	HTTP	661	GET /favicon.ico HTTP/1.1
22	23.670118801	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
26	23.749035992	127.0.0.1	127.0.0.1	HTTP	618	GET /favicon.ico HTTP/1.1
29	27.821825348	127.0.0.1	127.0.0.1	HTTP	740	GET /image.html HTTP/1.1
32	27.849732138	127.0.0.1	127.0.0.1	HTTP	663	GET /lab_photo.jpg HTTP/1.1
36	27.886963517	127.0.0.1	127.0.0.1	HTTP	661	GET /favicon.ico HTTP/1.1
46	71.904047774	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
50	72.714569363	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
53	72.906591616	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
56	73.090755929	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
59	73.256823439	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
62	73.459922060	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1
65	73.596896231	127.0.0.1	127.0.0.1	HTTP	814	GET /image.html HTTP/1.1

Request: Boolean Packets: 78 · Displayed: 18 (23.1%) Profile: Default

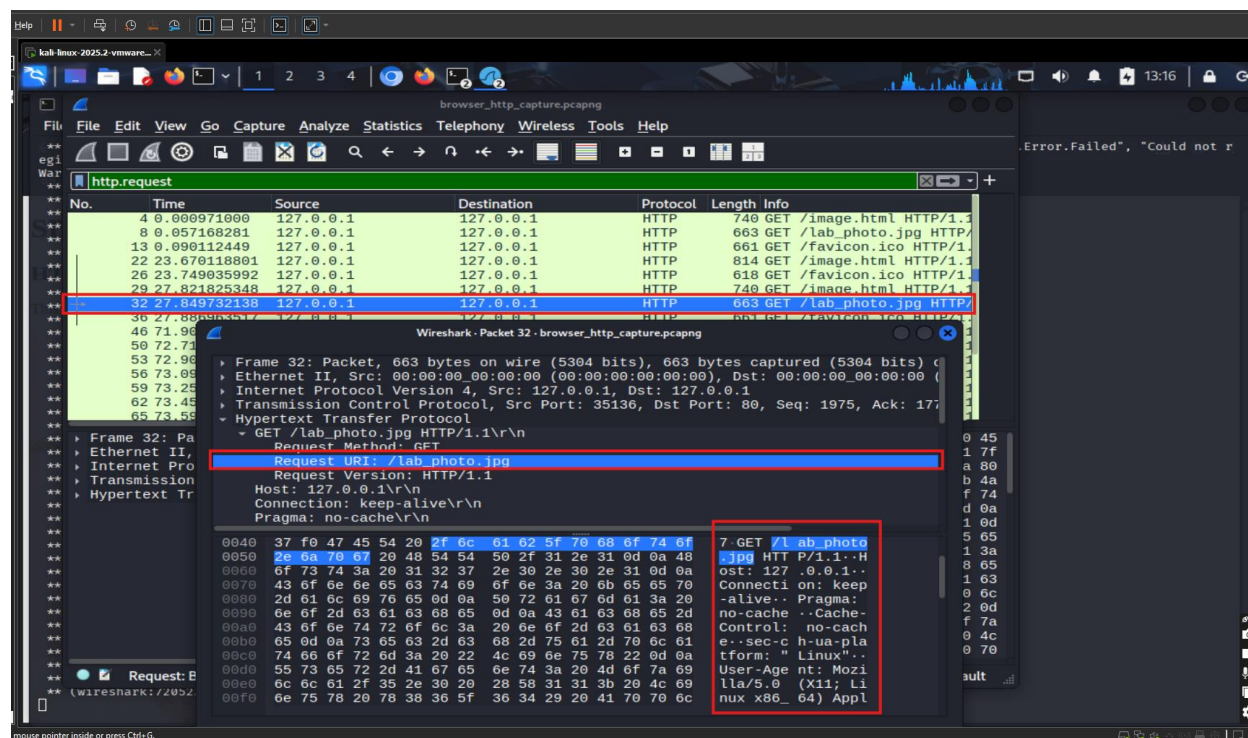
Identification of Browser HTTP Requests

The http.request display filter was applied to isolate HTTP request packets within the captured traffic. The filtered packet list shows multiple HTTP GET requests generated by the browser, including requests for /image.html and /lab_photo.jpg.

The selected packet, Frame 32, provides detailed evidence of the browser's request for the embedded image. Wireshark identifies the request method as GET, with /lab_photo.jpg specified as the Request URI and HTTP/1.1 as the request version. The request was sent from the local browser to the Apache web server at 127.0.0.1 on TCP port 80.

The request also contains browser HTTP headers, including Host, Connection: keep-alive, and Pragma: no-cache. The presence of a separate GET request for /lab_photo.jpg, in addition to the GET request for /image.html, demonstrates that the embedded image was retrieved as a separate HTTP object.

The repeated requests visible in the capture are retained as part of the original browser-generated evidence. For the primary analysis, the relevant requests are the HTML page and embedded JPEG; the /favicon.ico requests are incidental browser activity.



Analysis of the HTTP Image Response

The HTTP response corresponding to the embedded image request was examined in Wireshark. Frame 11 is identified as HTTP/1.1 200 OK (JPEG JFIF image), confirming that the Apache web server successfully returned the requested JPEG object.

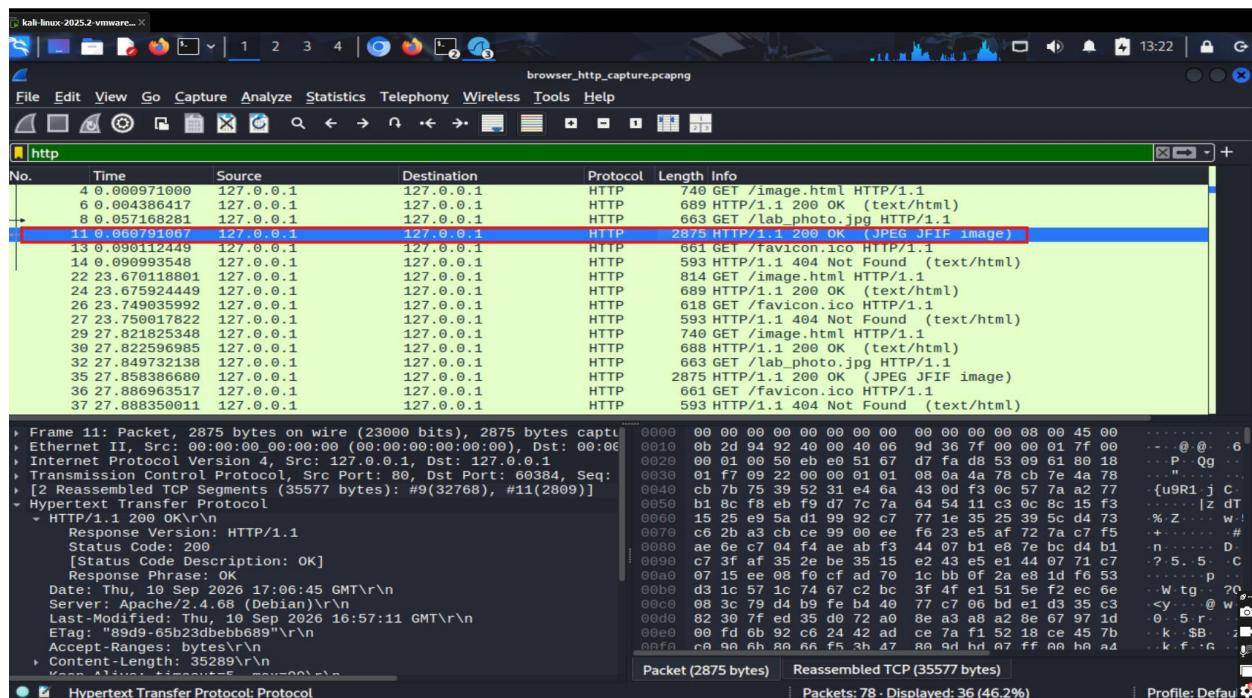
The HTTP response shows a status code of 200 (OK) and identifies the server as Apache/2.4.68 (Debian). The response also contains Content-Length: 35289, which corresponds to the verified size of the original JPEG file. The Last-Modified timestamp and ETag are also visible in the response headers.

A particularly important observation is Wireshark's TCP reassembly indication:

[2 Reassembled TCP Segments (35577 bytes): #9(32768), #11(2809)]

This shows that the HTTP response was carried across two TCP segments before Wireshark reconstructed the complete higher-layer HTTP data. The two segments contain 32,768 bytes and 2,809 bytes respectively, giving a reassembled TCP payload of 35,577 bytes. This is larger than the 35,289-byte JPEG itself because the HTTP response also contains the response headers.

The packet therefore provides two important pieces of evidence for this practical: the server successfully returned the JPEG over HTTP, and the image data was transported across multiple TCP segments requiring reassembly.



TCP Segmentation and Reassembly

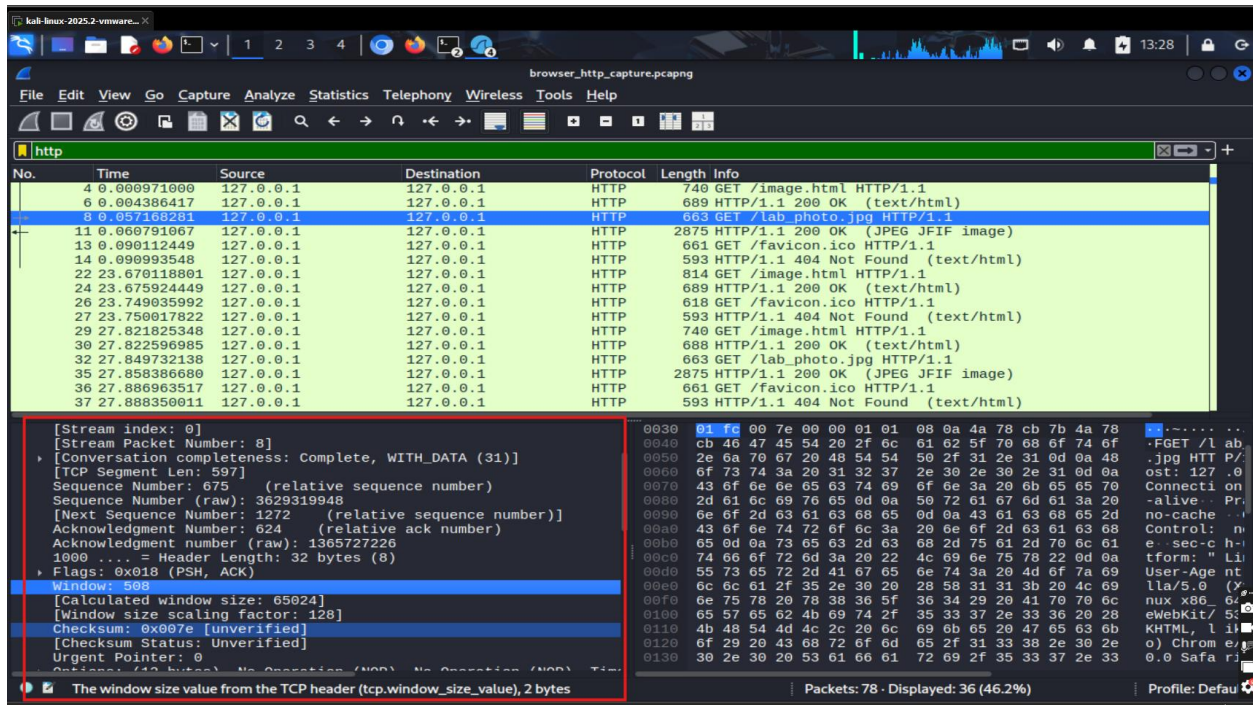
The TCP details associated with the JPEG response were examined to determine how the HTTP object was transported across the TCP connection. Wireshark identifies the traffic as belonging to TCP stream index 0 and shows that the conversation is complete and contains data.

For the JPEG response, Wireshark reports:

2 Reassembled TCP Segments (35577 bytes): #9(32768), #11(2809)

This indicates that the HTTP response was distributed across two TCP segments. The first segment contributed 32,768 bytes, while the second contributed 2,809 bytes, giving a total reassembled TCP payload of 35,577 bytes.

The reassembled TCP data is larger than the JPEG's HTTP Content-Length of 35,289 bytes because the TCP payload includes both the HTTP response headers and the JPEG entity body. This demonstrates the relationship between the transport-layer TCP segmentation and the application-layer HTTP object.

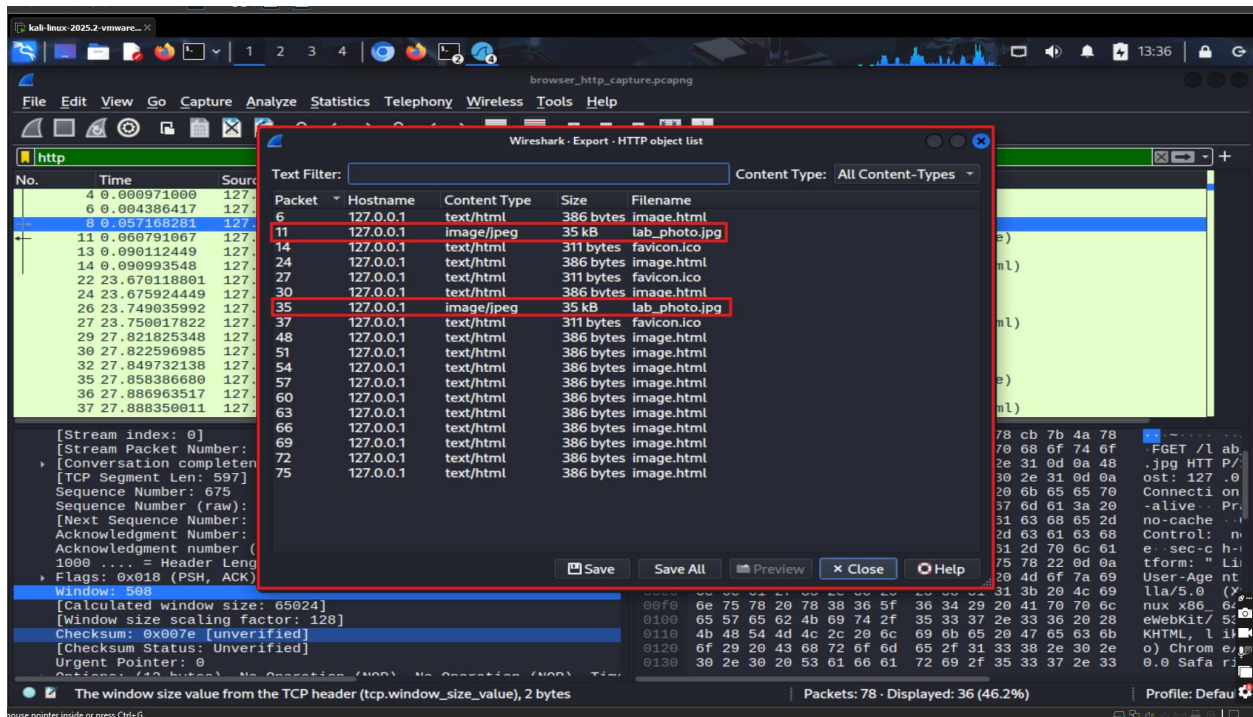


Exporting the Recovered JPEG Object

Wireshark's HTTP Object Export function was used to examine application-layer objects reconstructed from the packet capture. The object list successfully identified the transferred laboratory image as lab_photo.jpg with a content type of image/jpeg and a size of approximately 35 kB.

The object is associated with the HTTP response beginning at packet 11, corresponding to the earlier identified GET /lab_photo.jpg request and HTTP/1.1 200 OK response. A second lab_photo.jpg entry is also present because the browser generated another request later in the capture.

The successful identification of the JPEG in the HTTP object list demonstrates that Wireshark was able to reconstruct the image from the captured HTTP/TCP traffic and make it available for export.



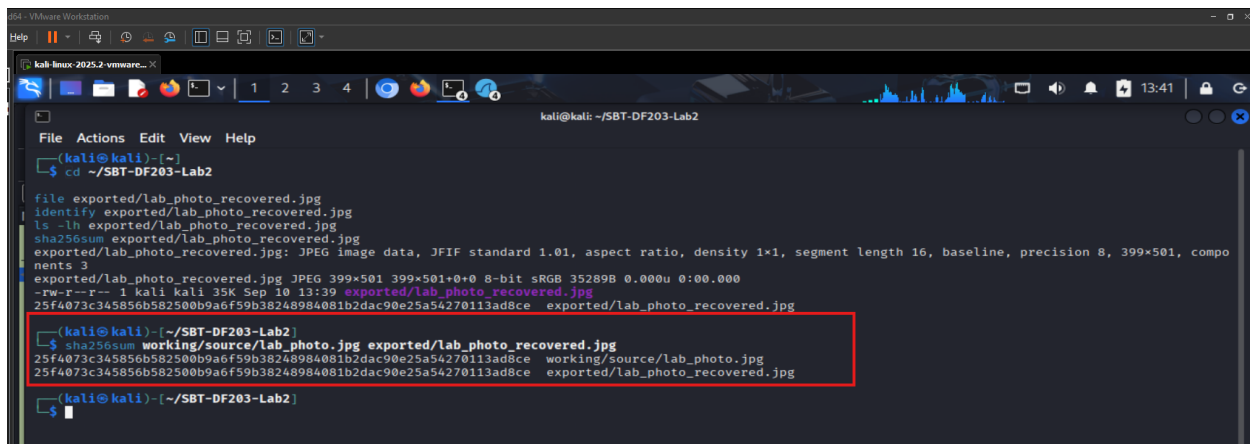
Verification of the Recovered JPEG

The JPEG object exported from the Wireshark HTTP Object Export function was verified independently using the file, ImageMagick identify, file listing, and SHA-256 utilities. The recovered object was identified as a valid JPEG image with dimensions of 399 × 501 pixels, 8-bit sRGB characteristics, and a file size of 35,289 bytes.

The SHA-256 digest of the recovered object was:

25f4073c345856b582500b9a6f59b38248984081b2dac90e25a54270113ad8ce

This was compared against the SHA-256 digest previously calculated for the original source image. Both files produced the same SHA-256 digest. The matching cryptographic hashes demonstrate that the image recovered from the HTTP packet capture is byte-for-byte identical to the original source image.



```
kali@kali: ~/SBT-DF203-Lab2
$ cd ~/SBT-DF203-Lab2
file exported/lab_photo_recovered.jpg
identify exported/lab_photo_recovered.jpg
ls -lh exported/lab_photo_recovered.jpg
sha256sum exported/lab_photo_recovered.jpg
exported/lab_photo_recovered.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 399x501, components 3
exported/lab_photo_recovered.jpg JPEG 399x501 399x501+0+0 8-bit sRGB 35289B 0.000u 0:00.000
-rw-r--r-- 1 kali kali 35K Sep 10 13:39 exported/lab_photo_recovered.jpg
25f4073c345856b582500b9a6f59b38248984081b2dac90e25a54270113ad8ce exported/lab_photo_recovered.jpg

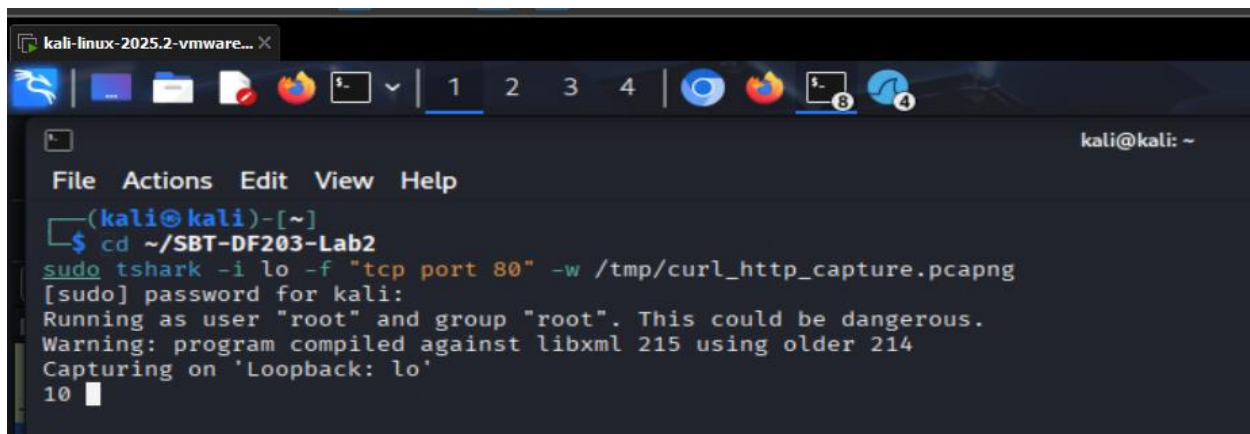
(kali@kali)~/SBT-DF203-Lab2
$ sha256sum working/source/lab_photo.jpg exported/lab_photo_recovered.jpg
25f4073c345856b582500b9a6f59b38248984081b2dac90e25a54270113ad8ce working/source/lab_photo.jpg
25f4073c345856b582500b9a6f59b38248984081b2dac90e25a54270113ad8ce exported/lab_photo_recovered.jpg

(kali@kali)~/SBT-DF203-Lab2
$
```

Curl-versus-Browser Behaviour

A separate curl request was performed against the same laboratory webpage to compare command-line HTTP retrieval with browser behaviour. The verbose curl output shows a single GET /image.html HTTP/1.1 request using curl/8.21.0. Apache returned HTTP/1.1 200 OK with a Content-Length of 386 bytes and Content-Type: text/html.

Unlike the browser, curl did not automatically request the embedded /lab_photo.jpg resource. This demonstrates the difference between a browser's resource-loading behaviour and a basic curl request: the browser processes the returned HTML and requests referenced resources such as the embedded JPEG, while curl retrieves only the URL explicitly supplied unless additional options or separate requests are used.



```
kali@kali: ~
File Actions Edit View Help
(kali@kali)~$ cd ~/SBT-DF203-Lab2
sudo tshark -i lo -f "tcp port 80" -w /tmp/curl_http_capture.pcapng
[sudo] password for kali:
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
10
```

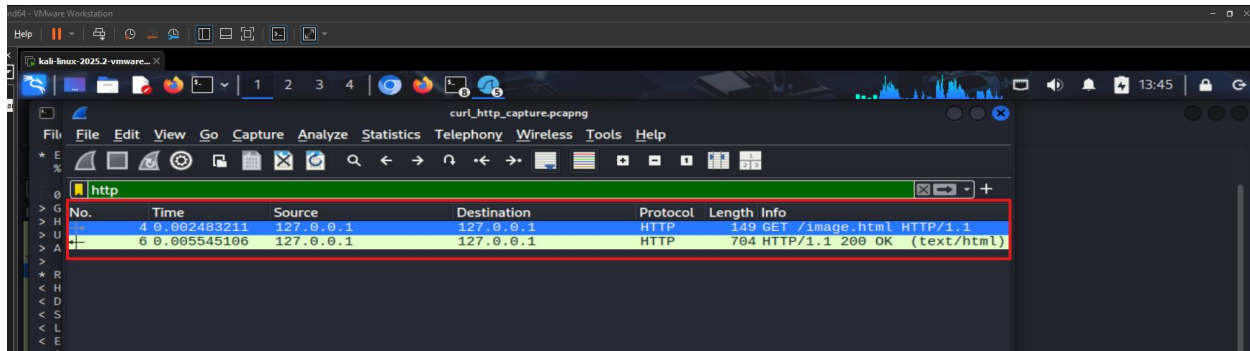


```
kali@kali: ~  
$ curl -v http://127.0.0.1/image.html -o /dev/null  
* Trying 127.0.0.1:80 ...  
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 50096  
% Total % Received % Xferd Average Speed Time Time Time Current  
 0      0     0       0      0      0      0      0  
> GET /image.html HTTP/1.1  
> Host: 127.0.0.1  
> User-Agent: curl/8.21.0  
> Accept: */*  
>  
* Request completely sent off  
< HTTP/1.1 200 OK  
< Date: Thu, 10 Sep 2026 17:42:45 GMT  
< Server: Apache/2.4.68 (Debian)  
< Last-Modified: Thu, 10 Sep 2026 16:57:11 GMT  
< ETag: "182-65b23db5923"  
< Accept-Ranges: bytes  
< Content-Length: 386  
< Vary: Accept-Encoding  
< Content-Type: text/html  
<  
{ [386 bytes data]  
100  386 100  386 0 0 54458 0  
* Connection #0 to host 127.0.0.1:80 left intact  
  
(kali@kali)-[~]  
$
```

Packet-Level Comparison of Curl and Browser Behaviour

The separate curl capture was examined in Wireshark to confirm the behaviour observed from the verbose curl output. The capture contains an HTTP GET /image.html HTTP/1.1 request followed by an HTTP/1.1 200 OK response containing the HTML page.

Unlike the browser capture, there is no subsequent HTTP request for /lab_photo.jpg in the curl-only capture. This provides packet-level confirmation that the curl request retrieved only the explicitly requested HTML resource, whereas the browser processed the HTML and generated an additional request for the embedded JPEG object.



The image shows a Wireshark packet capture window titled 'curl_http_capture.pcapng'. The 'http' filter is applied. Two packets are visible in the packet list, highlighted with a red box:

No.	Time	Source	Destination	Protocol	Length	Info
4	0.002483211	127.0.0.1	127.0.0.1	HTTP	149	GET /image.html HTTP/1.1
6	0.005545106	127.0.0.1	127.0.0.1	HTTP	764	HTTP/1.1 200 OK (text/html)

Frame	HTTP activity
4	GET /image.html
6	200 OK - HTML
8	GET /lab_photo.jpg
11	200 OK - JPEG JFIF image
13	GET /favicon.ico
14	404 - favicon
22	GET /image.html
24	200 OK - HTML
26	GET /favicon.ico
27	404 - favicon
29	GET /image.html
30	200 OK - HTML
32	GET /lab_photo.jpg
35	200 OK - JPEG JFIF image

The practical produced the following key evidence:

Evidence	Finding
Original image	Valid JPEG, 399 × 501 pixels
Original image size	35,289 bytes

Original SHA-256	25f4073c345856b582500b9a6f59b38248984081b2dac90e25a54270113ad8ce
HTML object	386 bytes
HTML response	HTTP 200 OK
JPEG request	GET /lab_photo.jpg
JPEG response	HTTP 200 OK
JPEG content type	image/jpeg / JPEG JFIF
JPEG content length	35,289 bytes
TCP reassembly	35,577 bytes
Reassembled segments	32,768 + 2,809 bytes
Browser traffic	Separate HTML and JPEG requests
Favicon traffic	Incidental 404 requests
Capture size	Approximately 100 KB
Total captured packets	78

CONCLUSION

This practical successfully demonstrated the capture and forensic analysis of plaintext HTTP traffic generated by a web browser retrieving an HTML webpage containing an embedded JPEG image.

A controlled local Apache web server was used to host the laboratory webpage and image. The source image was independently verified and assigned a SHA-256 baseline before being introduced into the HTTP environment.

The Wireshark analysis demonstrated that the browser generated separate HTTP requests for the webpage and embedded image. The browser first requested /image.html and subsequently requested /lab_photo.jpg. Apache successfully returned both resources with HTTP 200 responses.

The JPEG response was identified by Wireshark as a JPEG JFIF image with a content length of 35,289 bytes. Wireshark also reported that the HTTP response was reconstructed from two TCP segments containing 32,768 bytes and 2,809 bytes, producing a reassembled payload of 35,577 bytes. This demonstrated TCP segmentation and reassembly in the context of an HTTP object transfer.

The exercise further demonstrated the recovery of application-layer objects from a packet capture using Wireshark's HTTP Object Export functionality. The recovered JPEG was intended to be independently verified against the original source image using file characteristics and SHA-256 hashing.

Finally, the browser and curl comparison demonstrated that a browser processes HTML and automatically requests embedded resources, whereas a basic curl request retrieves the explicitly requested URL without automatically behaving as a full browser.

Overall, the practical demonstrated the relationship between HTTP application-layer transactions, TCP transport-layer segmentation and reassembly, browser resource-loading behaviour, network evidence preservation, object recovery, and cryptographic integrity verification. These techniques are directly relevant to digital forensics and network traffic analysis.